

TaskFlow Pro — Test Cases We Ran

Contata Hackathon 2026 — Testing & Reliability

How we approached testing

We built the dependency engine before we touched the API or the UI, mostly because it's the part that had to be right — a Kanban board with wrong dates underneath it is worse than useless. So the first thing we did once the engine had a rough shape was write pytest cases for it, and we kept adding to that suite every time we found (or suspected) a bug, rather than writing tests once at the end and hoping.

The list below is roughly in the order we wrote the tests, not by importance, because honestly the order tells its own story — you can see where we started worrying about something specific (the diamond case took us three attempts to get right, which is why there are two separate tests around it).

Engine and API test cases

Test case	What it checks	Result
Cycle rejection — direct ($A \rightarrow B \rightarrow A$)	Adding a link back to an ancestor is refused before it touches the DB	Pass
Cycle rejection — 3-node ($A \rightarrow B \rightarrow C \rightarrow A$)	Same check, deeper chain; error message returns the actual loop	Pass
Self-dependency ($A \rightarrow A$)	A task can't be its own prerequisite	Pass
Diamond, no double counting ($A \rightarrow B \rightarrow D$, $A \rightarrow C \rightarrow D$)	Extending A by 3 days should move D by 3, not 6	Pass
Multi-level propagation ($A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$)	A single change reaches every downstream task once, in order	Pass
Idempotent propagation	Running the recalculation twice in a row gives the same dates	Pass
Blocked $\rightarrow$ Ready transition	A task flips to Ready the moment its last open prerequisite is marked Done	Pass
Rollback on regression	Moving a Done task back to In Progress re-blocks everything downstream of it	Pass
Delete a task with dependents	Removing a task drops its edges and re-runs propagation for what's left	Pass
Duplicate dependency	Adding the same prerequisite twice is rejected, not silently ignored	Pass
Persistence across restart	Reloading the app after a restart shows the same graph and dates	Pass
Critical path on seed data	Longest chain by duration matches a hand-calculated expected path	Pass
Randomised DAG stress test	50 runs of random graphs, checked against a brute-force longest-path script	Pass
AI suggestion — unknown task id	A suggestion referencing an id we didn't send is dropped before display	Pass

AI suggestion — fabricated evidence	A suggestion whose quote isn't actually in the task text is dropped	Pass
AI suggestion — would create a cycle	Passed through the same cycle check as manual edges; rejected the same way	Pass
AI suggestion — duplicate of existing edge	Filtered out so the review list doesn't get cluttered with edges we already have	Pass

How to reproduce these results

From the repo root:

- `cd api && pip install -r requirements.txt`
- `pytest -v` # runs the full suite above
- `python scripts/eval_suggestions.py` # prints precision/recall for the AI suggestion filter against our seed data's ground-truth edges
- `python scripts/random_dag_check.py --runs 50` # the randomised stress test, run standalone since it's slower than the rest of the suite

What we didn't get to

We didn't write end-to-end browser tests (Playwright/Cypress) — everything above is backend and engine level. We tested the frontend manually, by clicking through the flows ourselves and with one teammate who hadn't touched the code trying to break it. That's a real gap and it's the first thing we'd add with another day.

We also didn't load-test the API beyond the randomised DAG check, which exercises the engine but not concurrent requests hitting the same task at once. Section 4 of our known-failure document talks about what we expect would happen there.